

Sisteme de Operare

Cap 7. Primitive pentru sincronizarea proceselor

April 16, 2025

- 1 Condiții de concurență
- 2 Soluții hardware pentru sincronizare
- 3 Primitive software
- 4 Liveness failure

- 1 Condiții de concurență
- 2 Soluții hardware pentru sincronizare
- 3 Primitive software
- 4 Liveness failure

Race conditions

- procesele se pot executa concurent (time sharing, pe un singur core) sau în paralel (pe mai multe core-uri, simultan)
- considerăm două procese ce se execută concurent (sau paralel); ambele procese au acces la variabila comună `count`, cu valoarea 5
- procesul 1 execută `count++` în vreme ce procesul 2 execută `count--`
- rezultatul final poate fi 4, 5, sau 6; cel corect este 5, dacă instrucțiunile se execută serial (fiecare proces pe rând)
- instrucțiunile se implementează în limbaj mașină astfel:

```
// count++
```

```
reg1 = count
```

```
reg1 = reg1 + 1
```

```
count = reg1
```

```
// count--
```

```
reg2 = count
```

```
reg2 = reg2 - 1
```

```
count = reg2
```

- instrucțiunile din aceste procese sunt executate "întrețesut", nu există o garanție a punctelor unde se face comutarea între procese

Race conditions (2)

- o posibilă întrețesere:

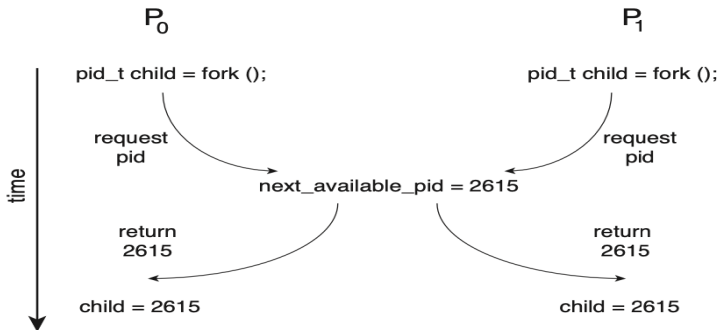
// proces 1: count++	// proces 2: count--	// rezultat
reg1 = count		// reg1 = 5
reg1 = reg1 + 1		// reg1 = 6
	reg2 = count	// reg2 = 5
	reg2 = reg2 - 1	// reg2 = 4
count = reg1		// count = 6
	count = reg2	// count = 4

- situațiile în care mai multe procese accesează concurent aceeași variabilă iar rezultatul execuției depinde de ordinea particulară în care au loc accesele alcătuiesc **condițiile de concurență**
- este necesar să ne asigurăm că doar un proces accesează variabila la un moment dat, adică să **sincronizăm** procesele

Problema secțiunii critice

- considerăm un sistem cu N procese, $P_1, P_2, \dots, P_{N-1}$; fiecare proces are un segment de cod denumit **secțiune critică** unde procesul accesează și modifică date partajate între toate cele N procese
- abilitatea specială furnizată de SO este ca atunci când unul dintre procese se află în secțiunea sa critică, nici un alt proces nu poate să se mai afle în secțiunea sa critică
- problema secțiunii critice constă în a proiecta un protocol prin care procesele să se sincronizeze în acest sens de acces exclusiv la datele partajate
- soluția problemei secțiunii critice trebuie să satisfacă:
 - ❶ **excluderea mutuală**: dacă procesul P_i își execută secțiunea critică, nici un alt proces nu se poate afla în execuția propriei secțiuni critice
 - ❷ **progres**: dacă nici un proces nu-și execută secțiunea critică și anumite procese vor să intre în secțiunea critică, se va selecta unul în timp finit
 - ❸ **așteptare limitată**: așteptarea procesului de a intra în secțiunea critică nu va dura la nesfârșit

Rezolvarea secțiunii critice



- o soluție ar fi dezactivarea întreruperilor, dar aceasta presupune sincronizarea tuturor core-urilor, propagarea acestui mesaj durează
- kernel-urile **non-preemptive** garantează că apelul de sistem nu este întrerupt, ca atare nu avem race conditions
- kernel-urile **preemptive** răspund într-un timp mai scurt aplicațiilor, există garanția că un proces aflat într-un apel de sistem lent va fi întrerupt; se pretează implementărilor de sistem în timp real

Soluția lui Peterson (codul pentru procesul i)

- este restrânsă la 2 procese, $i, j \in \{0, 1\}$, $i = 1 - j$, execuția se face alternând secțiunea critică cu restul codului din afara secțiunii critice

```
int turn, j = 1 - i;
boolean flag[2];
```

```
while (true) {
    flag[i] = true;
    turn = j;

    while (flag[j] && turn == j) ;
    /* critical section */

    flag[i] = false;
    /* restul codului din afara secțiunii */
}
```


Soluția lui Peterson (2)

- variabila `turn` arată cărui proces îi va fi permis să intre în secțiunea critică
- variabila `flag[i]` semnalează că procesul `i` este gata să intre în secțiunea critică
- doar unul va reuși să seteze ultima variabila `turn`
- fiecare proces intră în secțiunea critică doar dacă `flag[j] == false` sau `turn == i`, adică celălalt proces nu dorește să intre în secțiunea critică sau, dacă dorește, nu e rândul lui, ci al procesului `i`
- după ce iese din secțiunea critică, procesul `i` setează `flag[i] = false`, eliberând potențial celălalt proces din bucla infinită
- soluția nu funcționează dacă compilatorul reordonează instrucțiunile de atribuire care nu au dependențe între ele

- 1 Condiții de concurență
- 2 Soluții hardware pentru sincronizare**
- 3 Primitive software
- 4 Liveness failure

Bariere de memorie

- modelele de memorie pentru multi-procesoare depind de arhitectura de calcul:
 - ① **strongly ordered**, unde o modificare a memoriei de către un procesor este imediat vizibilă celorlalte procesoare;
 - ② **weakly ordered**, modificarea nu este imediat vizibilă celorlalte procesoare.
- arhitectul hardware furnizează instrucțiuni care forțează propagarea schimbărilor de memorie, denumite **memory barriers**; la execuția sa, sistemul se asigură că toate accesele la memorie sunt complete înainte de a efectua alte accese la memorie
- barierele de memorie sunt considerate operații low-level, folosite doar de dezvoltatorii kernel-ului SO pentru scrierea codului care asigură excludere mutuală

Instrucțiuni hardware

- asigură accesul la o locație de memorie pentru verificarea și modificarea într-o singură operație, **atomică** (în sensul de indivizibilă)
- se folosește pentru rezolvarea problemelor de secțiuni critice

```
boolean test_and_set(boolean *target) {
    boolean rv = *target;
    *target = true;
    return rv;
}
```

- implementarea secțiunii critice:

```
do {
    while (test_and_set(&lock)) ; /* do nothing */
    /* critical section */

    lock = false;
    /* remainder section */
} while (true);
```

Instrucțiuni hardware: `compare_and_swap`

- folosește un mecanism diferit bazat pe interschimbarea conținutului

```
int compare_and_swap(int *val, int expected, int new_val)
{
    int temp = *val;
    if (*val == expected)
        *val = new value;
    return temp;
}
```

- implementarea secțiunii critice:

```
while (true) {
    while (compare_and_swap(&lock, 0, 1)) ; /* do nothing
    /* critical section */

    lock = 0;
    /* remainder section */
}
```

Instrucțiuni hardware: bounded waiting

- este posibil ca un proces să aștepte nedefinit să intre în secțiunea critică, deoarece celălalt poate fi suficient de rapid; sau sunt mai multe procese, iar unul "reușește" să nu intre deloc în secțiunea sa critică
- va trebui să construim un algoritm care oferă garanții: odată ce un proces și-a manifestat intenția de a intra în secțiunea critică, un alt proces care își manifestă ulterior aceeași intenție va trebui să intre în secțiunea critică după primul proces
- vom folosi un array `waiting` care conține informația de stare a proceselor care vor să intre în secțiunea critică
- dacă există procese care așteaptă, procesul care părăsește secțiunea critică va desemna următorul proces care intră în secțiunea critică

Bounded waiting cu compare_and_swap

```
while (true) {  
    waiting[i] = true;  
    key = 1;  
    while (waiting[i] && key == 1)  
        key = compare_and_swap(&lock, 0, 1);  
    waiting[i] = false;  
    /* critical section */  
    j = (i + 1) % n;  
    while ((j != i) && !waiting[j])  
        j = (j + 1) % n;  
    if (j == i)  
        lock = 0;  
    else  
        waiting[j] = false;  
    /* remainder section */  
}
```

Variabile atomice (indivizibile)

- instrucțiunea `compare_and_swap` nu este folosită direct pentru furnizarea accesului exclusiv, ci stă la baza altor primitive ce rezolvă problema secțiunii critice
- **variabila atomică** furnizează operații atomice pe tipurile de date primitive (`int`, `boolean`); se folosește pentru incrementarea/decrementarea în siguranță a variabilelor partajate între procese
- exemplu - incrementarea unei secvențe de întregi:

```
void increment(atomic int *v) {  
    int temp;  
    do { temp = *v; }  
    while (temp != compare_and_swap(v, temp, temp + 1));  
}
```


- 1 Condiții de concurență
- 2 Soluții hardware pentru sincronizare
- 3 Primitive software**
- 4 Liveness failure

Mutex locks

- dezvoltatorii SO furnizează unelte software pentru rezolvarea problemei secțiunii critice; cea mai simplă este un **mutex lock** (MUTual EXclusion)
- un mutex lock va prelua lock-ul la intrarea în secțiunea critică și îl va elibera la ieșire; un proces care nu reușește să preia lock-ul se va bloca până când lock-ul va deveni din nou disponibil
- un lock are o variabilă denumită *available* ce indică dacă lacătul este sau nu disponibil
- definițiile celor două funcții de la intrarea și ieșirea din secțiunea critică sunt:

```

acquire() {                                |   release() {
    while (!available) ;                    |       available = true;
    available = false;                     |   }
}                                           |

```

- apelurile `acquire()` și `release()` trebuie să fie atomice, se implementează cu funcția `compare_and_swap`

Spinlocks

- **lock contention**: exprimă numărul de thread-uri (processe) care încearcă să preia acel lock; high contention degradează performanțele aplicațiilor concurente
- dezavantajul major al soluției acquire și release este aceea că necesită **busy waiting**, care irosește tacte CPU ce nu sunt folosite pentru lucruri utile
- procedeul este denumit **spinlock**, procesul așteptând activ eliberare lock-ului, fără a face comutări de context
- **spinlock** este metoda preferată dacă lock-ul nu este preluat decât pentru foarte scurt timp, deoarece comutarea de context care ar apărea altfel este mult mai costisitoare ca număr de instrucțiuni

Semafoare

- mutex-urile sunt cele mai simple dintre primitivele de sincronizare
- **semaforul** este o variabilă întreagă care este accesată doar de primitivele `wait()` și `signal()`

```

wait(S) {                               |  signal(S) {
    while (S <= 0) ;                     |      S++;
    S --;                               |  }
}                                       |

```

- toate operațiile `wait` și `signal` se execută atomic
- un semafor binar (care poate lua doar valorile 0 și 1) funcționează la fel ca un mutex
- semaforul se inițializează cu o valoare strict pozitivă, egală cu numărul resurselor pe care le controlează; când ajunge la 0, toate resursele au fost deja folosite

Implementarea semafoarelor (1)

- la un apel `wait()`, procesul care găsește că valoarea semforului nu e pozitivă, trebuie să aștepte, nu prin busy waiting, ci suspendându-se
- suspendarea plasează procesul în coada de **wait**
- un proces suspendat pe baza unui semafor este trezit de un apel `signal()` și plasat din nou în coada **ready**
- SO va decide ce proces din lista de procese care așteaptă pe acel semafor, este trezit (waked up)
- structura de date folosită:

```
typedef struct {  
    int value;  
    struct process *list;  
} semaphore;
```

Implementarea semafoarelor (2)

- implementarea primitivelor:

```
wait(semaphore* S) {  
    S->value--;  
    if (S->value < 0) {  
        add this process to S->list;  
        sleep();  
    }  
}  
  
signal(semaphore* S) {  
    S->value++;  
    if (S->value <= 0) {  
        remove a process P from S->list;  
        wakeup(P);  
    }  
}
```

Implementarea semafoarelor (3)

- apelul de sistem `sleep()` suspendă execuția procesului care l-a invocat
- apelul `wakeup(P)` rezumă execuția procesului P care se găsește într-o stare suspendată
- în această implementare, valoarea unui semafor poate deveni negativă, însă în implementarea clasică a semaforului, valoarea acestuia nu poate fi decât pozitivă sau zero
- procedurile `wait` și `signal` sunt implementate atomic prin realizarea unei secțiuni critice, cu ajutorul implementărilor precum `compare_and_swap` sau spinlock-urile
- busy waiting-ul nu a fost eliminat complet, ci a fost restrâns doar pentru secțiunile critice ale lui `wait` și `signal`, care sunt foarte scurte, nu mai mult de 10 instrucțiuni procesor; secțiunea critică în acest mic caz este rareori găsită ocupată, și doar pentru scurt timp (se justifică folosirea spinlock-urilor)

Monitoare

- utilizarea incorectă a semafoarelor duce la erori de sincronizare greu de detectat și de reprodus, datorită caracterului nedeterminist
- de exemplu, un proces poate inversa ordinea primitivelor, apelând întâi `signal()` și apoi `wait()`; pot exista astfel mai multe procese simultan în secțiunea critică
- o altă posibilitate este ca secțiunea critică să înceapă cu `wait()` și să se termine tot cu `wait()`; procesul se va bloca permanent în al doilea apel `wait`
- dacă procesul uită să apeleze una din primitive, fie vom avea mai multe procese simultan în secțiunea critică, fie procesul se va bloca
- **monitorul** este o construcție de nivel înalt care folosește semaforul ca primitivă de sincronizare
- furnizează un set de funcții care sunt mutual exclusive, și care accesează doar variabilele locale declarate de monitor

Monitorul ca tip de date abstract

```
monitor monitor name {  
    /* shared variable declarations */  
    function P1 ( . . . ) {  
        . . .  
    }  
    function P2 ( . . . ) {  
        . . .  
    }  
  
    function Pn ( . . . ) {  
        . . .  
    }  
    initialization code ( . . . ) {  
        . . .  
    }  
}
```

Monitor pentru alocarea unei singure resurse

```
monitor monitor name {  
    boolean busy;  
    condition x;  
    void acquire(int time) {  
        if (busy)  
            x.wait(time);  
        busy = true;  
    }  
    void release() {  
        busy = false;  
        x.signal();  
    }  
    initialization code() {  
        busy = false;  
    }  
}
```

Reluarea proceselor din monitor

- strategia implicită, dacă o serie de procese sunt suspendate pe o condiție x și o operație $x.\text{signal}()$ este executată de un alt proces, este să se aleagă un proces prin metoda FCFS, astfel ca procesul care așteaptă de cel mai mult timp să fie reluat primul
- **conditional waiting** folosește un întreg care asignează o prioritate c procesului care se va suspenda:

`x.wait(c);`

- la reluare, procesul cu cel mai mic **priority number** c va fi restaurat (resumed)

- 1 Condiții de concurență
- 2 Soluții hardware pentru sincronizare
- 3 Primitive software
- 4 Liveness failure

Deadlocks

- **liveness** este proprietatea ca un process să realizeze progres în timp ce se execută
- un proces nu are progres dacă așteaptă la infinit accesul la o resursă (liveness failure)
- sunt mai multe tipuri de liveness failure, toate caracterizate de performanță slabă și timp de răspuns mare
- așteptarea unui eveniment care poate fi cauzat numai de către un proces care este deja blocat se numește **deadlock**

P_0	P_1
wait(S);	wait(Q);
wait(Q);	wait(S);
.	.
.	.
.	.
signal(S);	signal(Q);
signal(Q);	signal(S);

Inversarea priorităților

- apare când un proces de prioritate ridicată trebuie să citească o structură kernel accesată și blocată de un proces de prioritate scăzută
- structura kernel fiind protejată de un lock, procesul de prioritate ridicată va aștepta după procesul de prioritate scăzută, care s-ar putea să nu ajungă la rulare din cauza altor procese de prioritate mai mare
- de exemplu, avem trei procese ordonate după priorități, $L < M < H$ și presupunem că H este suspendat într-un semafor blocat de procesul L; procesul L nu se poate executa din cauza procesului M de prioritate mai mare; procesul M de prioritate mai mică ajunge, indirect, să blocheze procesul H de prioritate mai mare - **priority inversion**
- se evită prin implementarea protocolului **priority inheritance**: procesele ce accesează resurse dorite de procese de prioritate mai mare, vor moșteni prioritatea mai mare până la terminarea accesului la acele resurse partajate, apoi prioritatea lor revine la normal
- procesul L se va executa cu prioritatea lui H în accesarea resursei, M nu mai este planificat, iar la terminarea accesului se va rula din nou H

Unelte de sincronizare

- soluțiile hardware de tipul Compare And Swap (CAS) sunt considerate o abordare **optimistă**, mai întâi se modifică variabila și apoi se detectează conflictul; operația este reîncercată repetat până când reușește fără conflict
- excluderea mutuală folosind mutex sau semafoare este o strategie **pesimistă**; se presupune că un alt thread actualizează variabila și se solicită obținerea lock-ului înaintea actualizării
- următoarele situații sunt posibile:
 - ① **uncontended**¹: ambele sunt rapide, CAS însă ceva mai rapid
 - ② **contention moderat**: CAS va fi mult mai rapid decât sincronizarea clasică
 - ③ **contention ridicat**: în cazul unei concurențe ridicate la resursă, sincronizarea clasică cu mutex-uri sau semafoare este mult mai rapidă decât mecanismul CAS, acele spinlock-uri pentru care se va face și comutare de context vor încărca CPU-ul cu cicluri busy-waiting

¹tr. "fără concurență"

Contention

- în cazul concurenței moderate, spinlock-urile vor fi executate relativ rapid, față de situația semafoarelor care vor provoca comutarea contextului și plasarea threadului în coada de așteptare (wait queue)
- alegerea mecanismului pentru tratarea condițiilor de concurență afectează masiv performanța; astfel, atomic integers au impact mult mai mic decât lock-urile tradiționale, și sunt preferate pentru modificarea variabilelor partajate precum counter-ele
- pentru numărarea accesului la resurse limitate, vom prefera folosirea semafoarelor
- construcțiile de nivel înalt precum monitoarele au un overhead ridicat și pot înrăutăți performanțele în cazul unei concurențe ridicate la resursă (multe thread-uri care apelează aceeași primitivă / apel de sistem de tip monitor)